Word Embedding'lere Giriş: Kelimeleri Sayıdan Anlama Taşımak

Atıl Samancıoğlu

1 Bu Döküman Neden Var?

Transformer, Attention veya hatta RNN konuşmadan önce çok kritik bir soruyu netleştirmemiz gerekiyor:

> "Bir model için kelime ne demektir?"

Bilgisayarlar için kelimeler:

- Harf değildir
- Anlam değildir
- Sadece sayıdır

Ama **hangi sayı** olduğu her şeyi değiştirir.
Bu dökümanın amacı:

- Kelimeleri sayıya çevirmenin neden zor olduğunu
- Word embedding fikrinin neden ortaya çıktığını
- Word2Vec'in bu problemi nasıl çözdüğünü

adım adım ve sezgisel olarak anlatmak.
Bu döküman, Transformer serisinin **doğal başlangıç noktasıdır**.

2 En Naif Yaklaşım: Kelimeleri Etiketlemek

İlk akla gelen fikir şudur:

> "Her kelimeye bir numara verelim."

Örneğin:

Kelime	ID
dog	1
cat	2
apple	3
car	4

Bu yaklaşımın problemi çok nettir:

- 4, 1'den büyük diye "car", "dog"dan büyük değildir
- Sayılar arasında anlamsal ilişki yoktur

Model için bu sadece **etikettir**.

3 One-Hot Encoding: Bir Adım İleri

Bir sonraki fikir şudur:

> "Her kelimeyi bir vektörle temsil edelim."

Örneğin kelime dağarcığımız 5 kelimeyse:

$$\text{dog} = [1, 0, 0, 0, 0] \quad \text{cat} = [0, 1, 0, 0, 0]$$

Bu yaklaşım şunu çözer:

- Sayısal temsil var
- Model bu vektörlerle çalışabilir

Ama çok ciddi bir problem ortaya çıkar.

3.1 One-Hot Neden Kötü?

One-hot vektörlerde:

- Tüm kelimeler birbirine eşit uzaklıktadır
- "dog" ile "cat" arasındaki mesafe, "dog" ile "car" arasındakiyle aynıdır

Yani model için:

> "dog cat car" dışında hiçbir bilgi yoktur.

Anlam yoktur.

4 Kritik Fikir: Anlamı Geometrik Hale Getirmek

Burada çok önemli bir fikir ortaya çıktı:

> "Benzer kelimeler, uzayda birbirine yakın olsun."

Yani:

- "dog" ve "cat" yakın
- "dog" ve "car" uzak

Bu fikir bizi **word embedding** kavramına götürür.

5 Word Embedding Nedir?

Word embedding şudur:

> Kelimeleri, anlam ilişkilerini yansıtan yoğun (dense) vektörler olarak temsil etmek.

Bu vektörler:

- Düşük boyutludur (örneğin 100, 300)
- Öğrenilir
- Anlam taşır

Ama burada kritik soru şudur:

6 Bu Vektörleri Kim Öğretiyor?

Cevap:

> "Dil verisinin kendisi."

Bu bizi Word2Vec'e getirir.

7 Word2Vec'in Temel Sezgisi

Word2Vec şu basit varsayıma dayanır:

> "Benzer bağlamlarda geçen kelimeler, benzer anlamlara sahiptir."

Bu cümle çok önemlidir. Çünkü Word2Vec **etiketli veri kullanmaz**.
Sadece şunu yapar:

- Cümlelere bakar
- Kelimelerin komşuluklarını öğrenir

8 İki Farklı Word2Vec Modeli

Word2Vec'in iki ana varyantı vardır:

8.1 CBOW (Continuous Bag of Words)

CBOW şunu sorar:

> "Bu kelimeler etrafındaysa, ortadaki kelime ne olabilir?"

Örnek:

> *"I did __ say he stole"*

Modelin görevi:

not

8.2 Skip-Gram

Skip-Gram tam tersini yapar:

> "Bu kelime verildiyse, çevresinde hangi kelimeler olur?"

Örnek:

$$\text{Input: not} \rightarrow \text{Output: did, say}$$

Bu yaklaşım özellikle:

- Nadir kelimelerde
- Büyük veri setlerinde

çok etkilidir.

9 Matematiksel Ama Korkutmadan

Skip-Gram modelinde öğrenilen şey şudur:

$$P(w_{context} \mid w_{center})$$

Bu olasılık, embedding vektörleri üzerinden hesaplanır:

$$P(w_o \mid w_i) = \frac{\exp(v_{w_o}^\top v_{w_i})}{\sum_{w \in V} \exp(v_w^\top v_{w_i})}$$

Bu formül şunu yapar:

> "Bir kelimenin vektörüyle, başka bir kelimenin vektörü ne kadar uyumlu?"

Yani **nokta çarpımı = anlamsal uyum**.

10 Embedding Boyutları Ne Anlama Geliyor?

Şimdi en kritik zihinsel eşiğe geliyoruz.

Bir embedding vektörü şöyle görünür:

$$\text{king} = [0.21,\ -0.34,\ 0.87,\ \ldots,\ -0.12]$$

Bu sayılar:

- "erkeklik" değildir
- "krallık" değildir

Ama şunu temsil eder:

> "Bu kelime, bağlam uzayında hangi yönlere ne kadar yakındır?"

Yani embedding boyutları:

örtük (latent) anlamsal eksenlerdir.

11 Şimdi Asıl Sihri Açalım

Google Word2Vec makalesinde gözlemlenen şey şudur:

> Bazı anlamsal ilişkiler, vektör farkları olarak ortaya çıkar.

Örneğin:

$$\vec{king} - \vec{man} \approx \vec{queen} - \vec{woman}$$

Bu eşitliği **neden** bekleriz?

12 Sezgisel Açıklama (Çok Önemli)

Word2Vec şu istatistiği öğrenir:

- "king" ve "queen" benzer bağlamlarda geçer
- "man" ve "woman" benzer bağlamlarda geçer
- "king" ile "man" arasındaki fark, "queen" ile "woman" arasındaki farka benzer

Yani model şunu öğrenir:

> "Cinsiyet değişiyor ama rol korunuyor."

Bu, modele **özellikle öğretilmez**. Bağlam istatistiğinden **kendiliğinden** çıkar.

13 Bunu Somutlaştıralım (Tabloyla)

Aşağıdaki tablo **gerçek değerler değildir**, ama Word2Vec'in ne yaptığını sezgisel olarak **çok doğru** yansıtır.

Tablo 1: Word2Vec embedding'lerinde örtük anlamsal eksenler (temsili)

Kelime	**Kraliyet**	**İnsan**	**Erkeklik**	**Yetişkinlik**
king	0.95	0.90	0.98	0.85
queen	0.96	0.91	0.02	0.86
man	0.10	0.92	0.99	0.80
woman	0.11	0.93	0.01	0.81

Şimdi vektör farkına bakalım:

$$\text{king} - \text{man} \approx [+0.85,\ -0.02,\ -0.01,\ +0.05]$$

Bu fark:

> "Kraliyet ekle, erkekliği çıkar."

Aynı farkı “woman” üzerine uygularsak:

$$\text{woman} + (\text{king} - \text{man}) \approx \text{queen}$$

Bu yüzden şu işlem çalışır:

$$\vec{queen} \approx \vec{king} - \vec{man} + \vec{woman}$$

14 Bu Neden Lineer?

Bu çok kritik bir soru.

Cevap şu:

> Word2Vec, nokta çarpımı ve log-olasılık üzerinden optimize edildiği için anlamsal farklar lineer hale gelir.

Yani:

- Benzer bağlamlar → benzer yönler
- Farklılıklar → vektör farkları

Bu, Word2Vec'in **en büyük başarısıdır**.

15 Önemli Bir Uyarı

Bu tür cebirsel ilişkiler:

- Her kelime için çalışmaz
- Her veri setinde garanti değildir

Ama şunu kanıtlar:

> Anlam, vektör uzayında **geometrik olarak temsil edilebilir**.

Bu fikir olmasaydı:

- Attention anlamsız olurdu
- Transformer bu kadar güçlü olmazdı

16 Transformer Serisine Doğrudan Bağlantı

Transformer'daki embedding katmanı:

- One-hot değildir
- Word2Vec değildir

Ama şunu miras alır:

> “Anlam = vektör uzayında konum ve yön.”

Transformer, bu temsilleri **bağlama göre dinamik olarak** günceller.

17 Burada Durup Netleştirelim

Bu dökümanın sonunda şu net olmalı:

> Word embedding, NLP'de her şeyin başladığı yerdir.

Eğer kelimeyi doğru temsil edemezsen:

- Attention işe yaramaz
- Transformer mucize yaratmaz

Anlam, önce vektöre dökülür. Sonra model onu işler.

Artık kelimeler sayı değil, temsil.
Şimdi bu temsillerle neler yapabileceğimize bakabiliriz.